

Cryptography – Classical Encryption Techniques

Substitution ciphers

- Simplest: replace each plaintext letter by another letter
- More general variations

“what to substitute for each letter” is the enc/dec key

e.g. "a" : replace with symbol "Q"
"b" : replace with symbol "_"

→ Military, personal communications

Julius Caesar used a substitution cipher in which each letter in the plaintext is replaced by the letter three places further down in the alphabet (wrapping when necessary)

- “A” replaced by “D”; “B” replaced by “E”; ...; “Z” replaced by “C”
- now known as the “Caesar cipher”

Mathematically, we can represent this cipher in this way:

- Assign a number to each letter

A	B	C	D	E	F	G	H	I	J	K	L	M	N
0	1	2	3	4	5	6	7	8	9	10	11	12	13

O	P	Q	R	S	T	U	V	W	X	Y	Z
14	15	16	17	18	19	20	21	22	23	24	25

- Use the following equations (where $k=3$)

Two big problems with the Caesar cipher:

- There are only 26 possible keys
 - (see Stallings, Figure 2.3)

(25 really because key 0 is not used)

- Subject to frequency analysis

(Some letters in english are more frequently used than others).

HELLO JOHN	plaintext
DDDDDDDDDD	key(3)
K H O O R M R K Q	ciphertext

- In ciphertext, look for the most common letter.
This is probably the letter "E" (the most common letter).

- (see Stallings, Figure 2.5)

Monoalphabetic cipher (slight improvement over Caesar cipher)

- Instead of shifting each letter by the same amount, shift different letters by different amounts
- The key is therefore a string 26 letters in length

ABCDEFGHIJKLMNOPQRSTUVWXYZ

DKVQFIBJWPESCXHTMYAUOLRGZN → permutation of the alphabet

- Now, instead of only 26 keys (Caesar cipher), there are 26! keys (more than 4×10^{26} possible keys)
 - Rules out brute force searches, but does not solve the frequency analysis problem

➤ Cryptanalysis:

- use a table of letter frequencies, along with tables of double and triple letter frequencies. (Assume the attacker knows the language of the plaintext)

Example Cryptanalysis

- given the following ciphertext:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWP...

- count relative letter frequencies
- guess “P” & “Z” are “e” & “t”
- guess “ZW” is “th”, and hence “ZWP” is “the”
- the string “ZWSZ” is “th_t”, and so “S” is probably “a”
- proceeding with similar deductions (and some trial and error), we finally get:

it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the ...

Therefore, shifting each letter by the same amount does not seem to work, and shifting letters by different amounts does not seem to work

Why?

- Some of the structure of the plaintext survives in the ciphertext

One approach for dealing with this:

- Encrypt multiple letters at a time

Playfair cipher (encrypt digrams to digrams)

Create a 5x5 matrix of letters constructed using a keyword:

M	O	N	A	R
C	<u>H</u>	Y	<u>B</u>	D
E	F	G	I/J	K
L	<u>P</u>	Q	<u>S</u>	T
U	V	W	X	Z

In this example, *Keyword: "MONARCHY"*
All remaining letters in order
BD EF ...

Encryption is done two letters at a time, using a set of rules

- Arrange plaintext into digrams (adding "filler" letter as needed) *BALLOON*

digrams: ba lo on

- Rule for same row: **shift right**
e.g ar $\Rightarrow$ "RM"

- Rule for same column: **shift down**
mu $\Rightarrow$ "CM"

- Otherwise: use **same row** and **opposite column**
hs $\Rightarrow$ "BP"
ea $\Rightarrow$ "Im" (or "Jm")

Frequency analysis is more difficult than with monoalphabetic ciphers, but is not impossible.

— a few hundred letters of ciphertext will usually suffice

Note, too, that different keywords can leave portions of the key matrix unchanged.

↓
keywords of the same length

Hill cipher

Developed by mathematician Lester Hill in 1929

Encrypts m plaintext letters to m ciphertext letters.

For $m=3$, let the plaintext be $P = (p_1, p_2, p_3)$ and the ciphertext be $C = (c_1, c_2, c_3)$. Then the system can be described as a set of linear equations

$$\begin{aligned}c_1 &= (k_{11}p_1 + k_{12}p_2 + k_{13}p_3) \bmod 26 \\c_2 &= (k_{21}p_1 + k_{22}p_2 + k_{23}p_3) \bmod 26 \\c_3 &= (k_{31}p_1 + k_{32}p_2 + k_{33}p_3) \bmod 26\end{aligned}$$

$$\text{OR : } \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix}$$

More simply: $C = KP \bmod 26$

Decryption uses the inverse of K : $P = K^{-1}C \pmod{26}$
 (where $KK^{-1} = K^{-1}K = \text{the identity matrix } I$)

Cryptanalysis using frequency analysis is difficult, especially as m gets larger. This cipher hides single-letter frequencies, as well as digrams, trigrams, and so on, up to $(m-1)$ -grams, for any chosen block length m .

Thus, the Hill cipher is strong against a ciphertext-only attack. However, it falls easily (almost trivially) to a known-plaintext attack: Here $m=3$

Suppose we have m p/c pairs (of length m)
 $\therefore P_1 = \begin{pmatrix} p_{11} \\ p_{21} \\ \vdots \\ p_{m1} \end{pmatrix}$ $P_2 = \begin{pmatrix} p_{21} \\ p_{22} \\ \vdots \\ p_{m2} \end{pmatrix}$... $P_3 = \begin{pmatrix} p_{13} \\ p_{23} \\ \vdots \\ p_{m3} \end{pmatrix}$

Same for C_j
 $C_j = K P_j \pmod{26}$ for $1 \leq j \leq m=3$

Define 2 matrices $X = (P_{ij})_{m \times m} = 3 \times 3$

$Y = (C_{ij})_{m \times m} = 3 \times 3$

$\Rightarrow$ Then $Y = KX \pmod{26}$

If X has an inverse, then $K = YX^{-1} \pmod{26}$

and the key has been found.

$\therefore$ The cipher is easily broken as soon as we move to a stronger model of the attacker

Another approach (i.e., instead of encrypting multiple letters at a time) is to use multiple cipher alphabets

Polyalphabetic substitution ciphers

- Improve security over monoalphabetic substitution by using multiple cipher alphabets (therefore, ~~flatter~~ *flatter* ⁿ frequency distribution)
- Key selects which alphabet is used for each plaintext letter (repeat after end of key is reached)

Simplest example is Vigenere cipher

- Repeat keyword until key string is as long as the plaintext to be encrypted
- Use each key letter as a Caesar cipher key
- Makes frequency analysis more difficult

A = 0
B = 1
C = 2 ...

Example: key is "VIG" (Shift of 21, 8, 6)
(u) (i) (g)

H	E	L	L	O	J	O	H	N
V	I	G	V	I	G	V	I	G
C	M	R	G	W	P	J	P	T

$$\begin{matrix} H & + & V \\ (7) & & (21) \end{matrix} = 28 = 2 \bmod 26 = "C"$$

$$\begin{matrix} H & + & I \\ (7) & & (8) \end{matrix} = 15 = "P"$$

Cryptanalysis: make use of Babbage / Kasiski method (repetitions in ciphertext give clues to period) and/or Friedman method (Index of Coincidence), then attack each monoalphabetic cipher separately

Babbage/Kasiski method:

THE	BOY	HAS	THE	BAG
VIG	VIG	VIG	VIG	VIG
<u>OPK</u>	WWE	CIY	<u>OPK</u>	WIM

- plaintext blocks that encrypts to the same ciphertext blocks.
- These are likely to be a multiple of the keylength apart.

Friedman method:

- Compute the probability that 2 random ciphertext elements are the same letter

(see the example cryptanalysis of a Vigenere-encrypted ciphertext by Kranakis)

Intuition behind cryptanalysis of Viginere shown by Kranakis

ciphertext : D K J Z P A A ...
 1 2 3 4 5 6 ...

Friedman: We know the Index of Coincidence for english text = 0.065.
(IC)

This IC will hold if all letters are shifted by the same amount.

∴ Compute IC using all letters of the ciphertext (close to 0.038)
" " using every second letter (D J P A...) (close to 0.038)
" " using every third letter " "
" " using every fourth letter " "
" " using every fifth letter (close to 0.065)

Next step: Consider letters 1, 6, 11, 16 ... to be one string.
2, 7, 12, 17 ... another string.
3, 8, ...
4, 9, ...
5, ...

We know that strings 1 & 2 are shifted by different amounts

∴ Take every letter in string 2 and shift it by 1 letter.

Take MIC (Mutual Index of Confidence) between string 1 and this shifted string 2.

We repeat this process until we get a MIC of around 0.065

⇒ This would mean that we know the relative shift between

keyword letter 1 & keyword letter 2.

→ Do the same for strings 1 & 3, strings 1 & 4

→ Eventually, we will find all the relative shifts.

→ End up with: k_1 , $k_1 + \text{shift}_1$, $k_1 + \text{shift}_2$, $k_1 + \text{shift}_3$, $k_1 + \text{shift}_4$.

Exhaustive Search → Search for all values of k_1

Autokey cipher (want to avoid repeated keyword)

- Start with keyword
- Append plaintext

Example: THE BOY HAS THE BAG ...
VIG THE BOY HAS THE ...

Example: THIS IS THE PLAINTEXT
VIGT H I S I S T H E P L A I N T

Note: The decrypter does not need to know the plaintext to decrypt the ciphertext. (All he needs is the keyword, which would enable him to decrypt the first few letters, which in turn reveal how to decrypt the rest of the plaintext).

Benefit: reduces the risk of repeated ciphertext blocks.

??

Cryptanalysis: note that plaintext & key share the same frequency distribution of letters

∴ "E" encrypted with the letter "E" can be expected to occur with freq. $(0.127)^2 = 0.016$

→ Do this for all letters.

→ Make a frequency table of all letter products.

[A encryp. with A, A encryp. with B, A encryp. with C...]

So, the idea was right (i.e., a longer, non-repeating key), but the method was not ideal...

Vernam cipher (1918 – WW I)

- Choose a keyword that is as long as the plaintext and has no statistical relationship to it

Vernam proposed working on binary data rather than letters

$$C_i = P_i \oplus k_i$$
$$\therefore P_i = C_i \oplus k_i$$

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{array}$$

Vernam also proposed the use of a running loop of tape that eventually repeated the key, so in fact the system worked with a very long but repeating keyword.

- Therefore, can be broken given sufficient ciphertext

$$P_1 \oplus k = C_1$$
$$P_2 \oplus k = C_2$$
$$\text{Then } C_1 \oplus C_2 = P_1 \oplus P_2$$

Assume the attacker knows when the key repeats.

Mauborgne's improvement to Vernam cipher: random key;
as long as the plaintext; used only once

- ONE TIME PAD

This creates the world's only unbreakable cipher

(But, it has some practical limitations!)

Therefore, try to *simulate* the OTP: use a pseudorandom
key stream instead of a truly random key stream

- Modern examples include RC4 and Salsa20
- One of the original examples was the rotor machine

→ try to simulate the One-time pad

Rotor machine: (used in WW II)

A set of rotating cylinders (set up in the fashion of an odometer) through which electrical pulses can flow.

- Each cylinder has 26 input pins and 26 output pins, with internal wiring that connects each input to a unique output

(See Stallings, Figure 2.7)

With i rotors there are 26^i different substitution alphabets used before the system repeats. So, for 3, 4, or 5 rotors the machine uses 17,576, 456,976, and 11,881,376 alphabets, respectively. (A formidable polyalphabetic cipher!)

Given a machine, the key was the order of the cylinders and the starting position of each cylinder.

- The breaking of the German *Enigma* and the Japanese *Purple* codes was a significant factor in the Allies winning the war

A different approach to encryption: Transposition cipher
(permutation cipher)

- instead of substituting plaintext letters, perform a permutation on the plaintext letters

Ex: HELLO | THERE key is "53412"
 53412 | 53412
 OLLHE | EERTH
 "word scramble"

Not difficult to cryptanalyze:

- plaintext & ciphertext have identical letter frequencies (ciphertext letters are identical to plaintext letters).
- completely susceptible to a known or chosen plaintext attack.